Procedure Calls: Stack Use

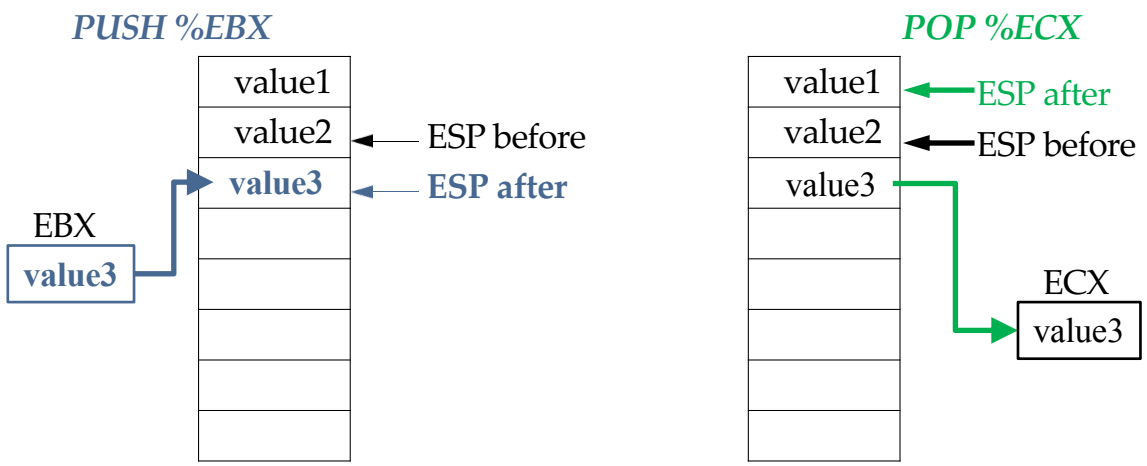
- CMPE310/CMSC313
- Dr. Kidd

Acknowledgements

- These lectures are duplicates of Dr. Kidd's CMPE310 lectures.
- These lectures are largely from Dr. Chintan Patel, UMBC, with significant modifications and additions from myself.

Purpose of Stack

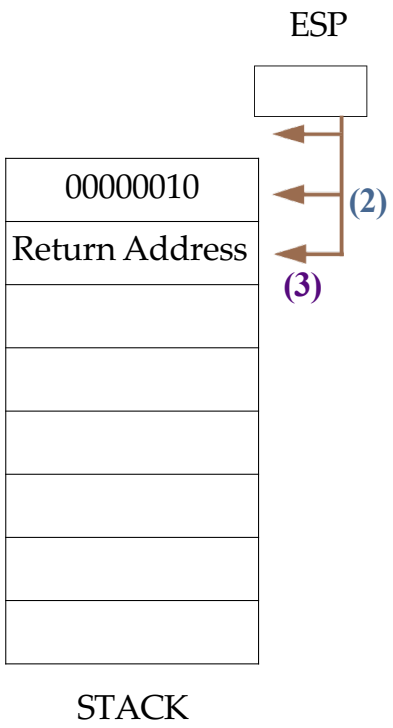
- Memory used to pass parameters to procedures (including C function calls)
- Memory used for allocating space for local variables
- Save return address in procedure calls
- Save registers to be preserved across procedure calls



Passing Parameters to Procedures

```
        .data
input_filename_ptr :
        .long 0                          (1)

        .text
main:
        pushd $input_filename_ptr        (2)
        call GetCommandLine              (3)
        add $4,%esp
```



(1) *input_filename_ptr :*

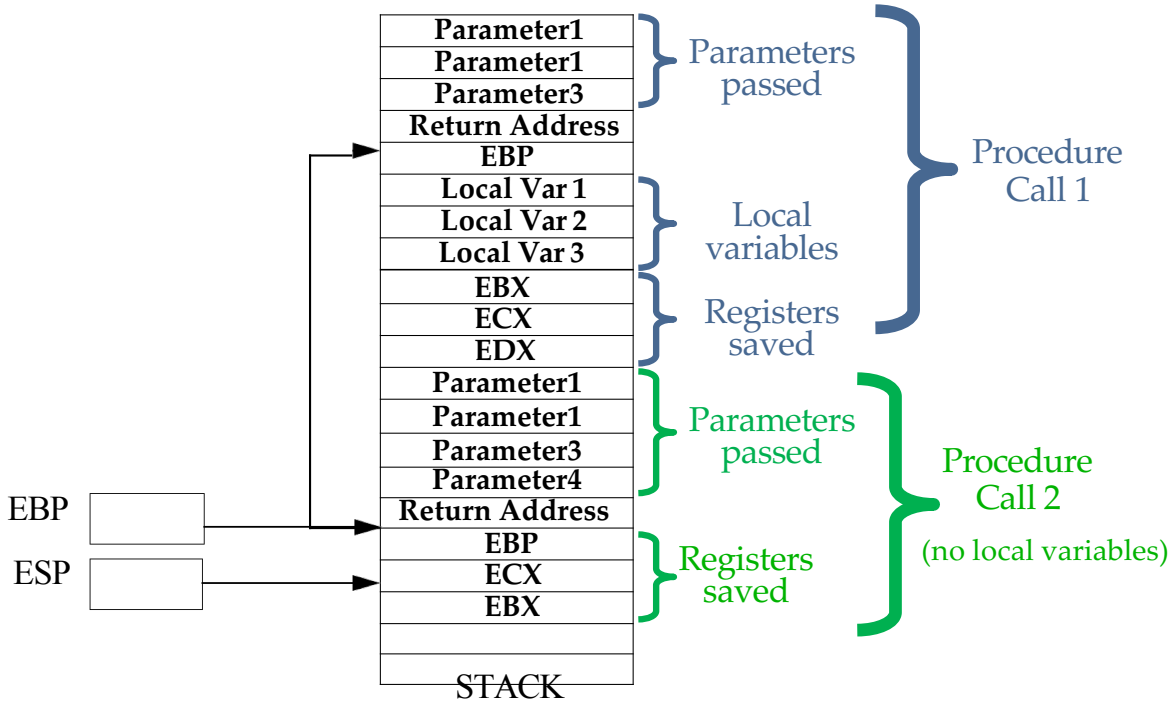
00000010 | Pointer to the filename

(2) Push the address of the pointer to the filename

(3) Return address pushed to the stack.
Address of the add instruction.

Call Frames

One call frame created per procedure call



Setting up Call Frames

```
GetCommandLine:
        Enter 0                    (1)
        Push_Regs ebx, ecx, edx    (2)
```

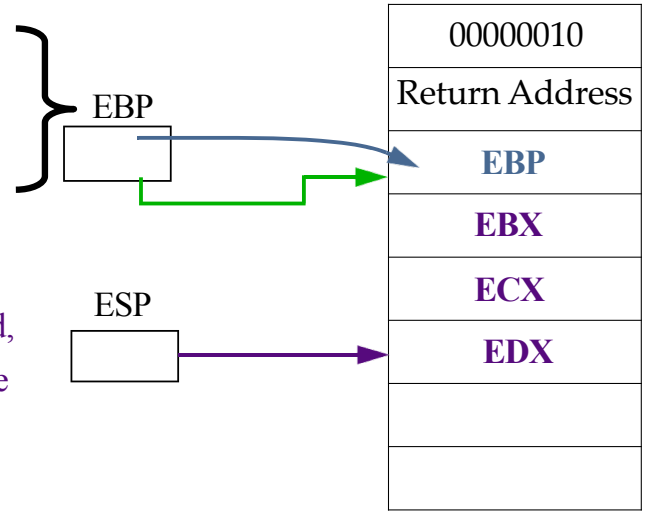
```
%macro  Enter 1
   push  %ebp
   mov   %esp,%ebp
   sub   $1,%esp
%endmacro
```

(1) ***Push* %ebp**

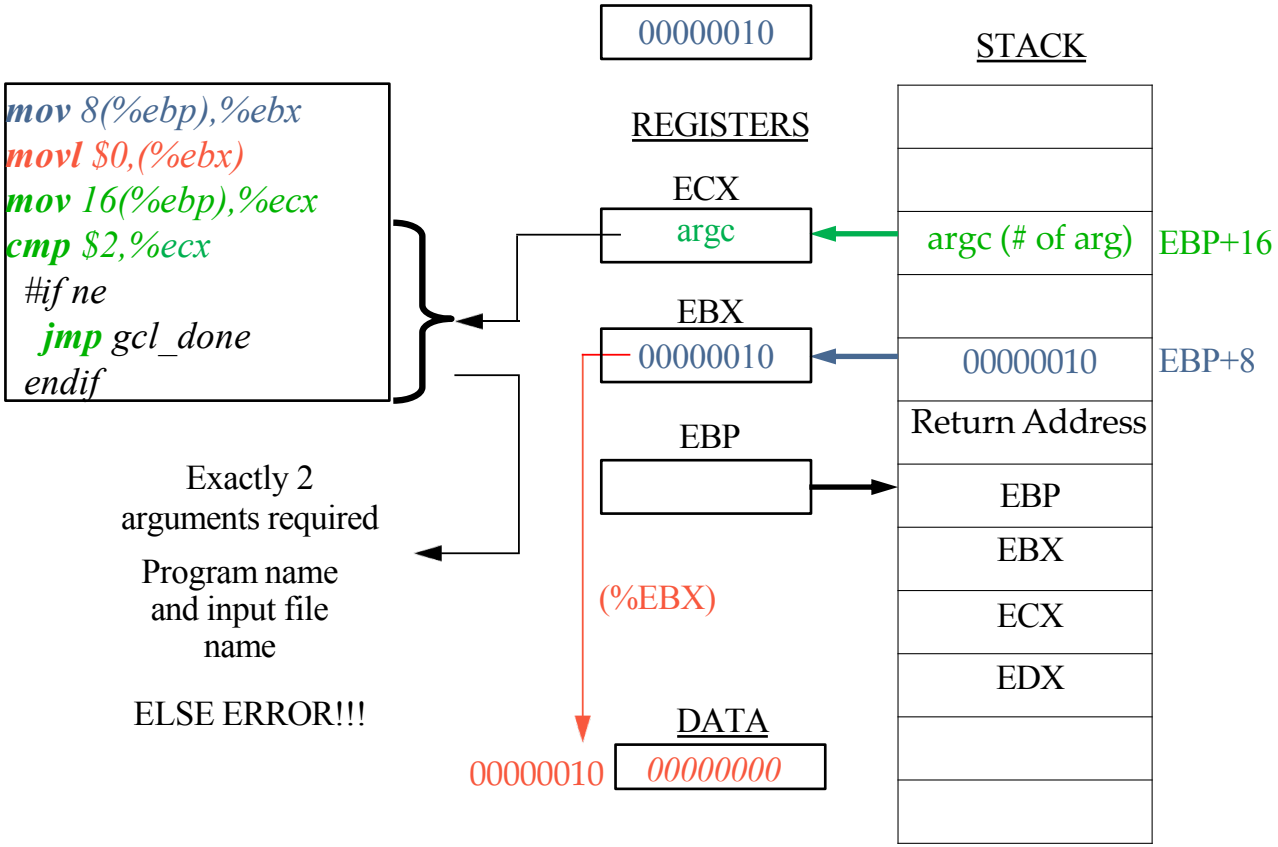
Move **ESP** into **EBP**
i.e. **EBP** points to the pushed **EBP**

Allocate space for local variables
(none in this example)

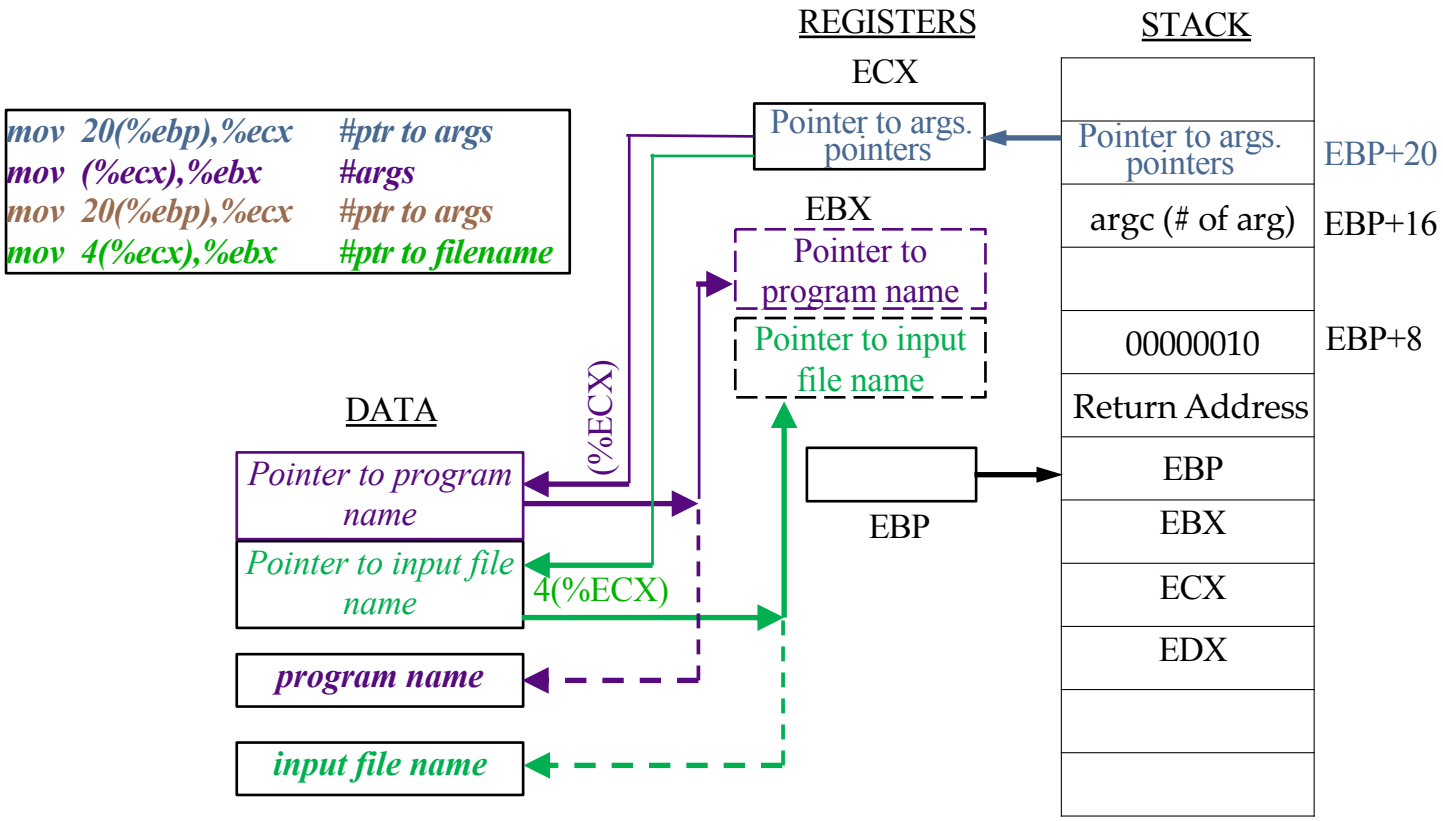
(2) Push the registers that are to be saved,
EBX, **ECX** and **EDX** in this example

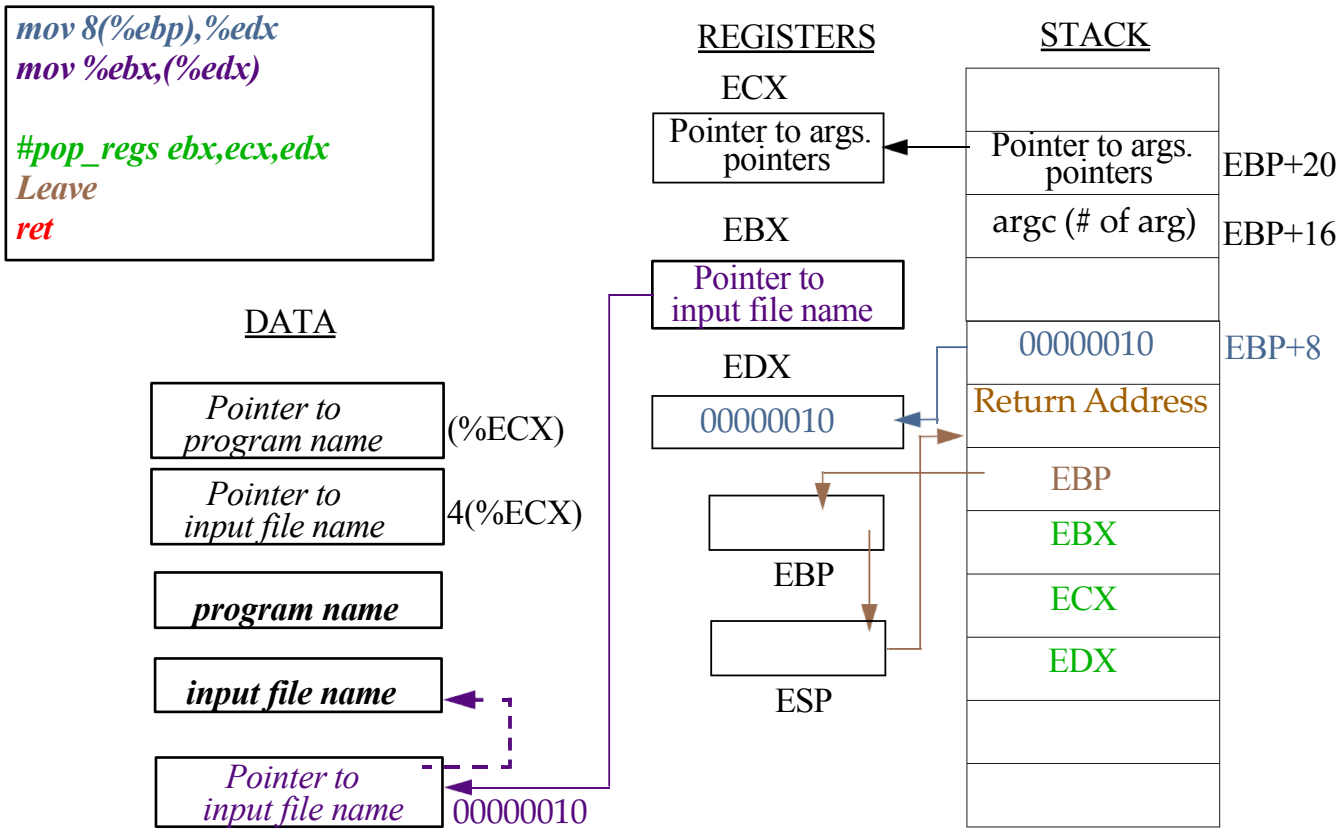


Reading Arguments



Reading Call




mov 8(%ebp),%edx
mov %ebx,(%edx)
#pop_regs ebx,ecx,edx
Leave
ret
DATA
Pointer to program name
(%ECX)
Pointer to input file name
4(%ECX)
program name
input file name
Pointer to input file name
00000010
REGISTERS
ECX
Pointer to args. pointers
EBX
Pointer to input file name
EDX
00000010
EBP
ESP
STACK
Pointer to args. pointers
EBP+20
argc (# of arg)
EBP+16
00000010
EBP+8
Return Address
EBP
EBX
ECX
EDX

Procedure Calls (Steps Recap)

- Caller: Before Call
  - Save registers that are needed (for C functions save **EAX**, **ECX**, **EDX**)
  - Push arguments, last first
  - ***CALL*** the function

- Callee:
  - Save caller's **EBP** and set up callee stack frame (ENTER macro)
  - Allocate space for local variables and temporary storage
  - Save registers as needed (C functions save **EBX**, **ESI**, **EDI**)
  - Perform the task
  - Store return value in **EAX**
  - Restore registers (C functions restore **EBX**, **ESI**, **EDI**)
  - Restore caller's stack frame (LEAVE macro)
  - Return

- Caller: After Return
  - ***POP*** arguments, get return value in **EAX**, restore registers (for C **EAX**, **ECX**, **EDX**)